Auditable Algorithmic Reasoning via Contract-Grounded Traces

Anonymous Author(s)
Affiliation
Address
email

Abstract

Large language models can solve many algorithmic questions, but their free-form reasoning traces are difficult to inspect, compare, and validate. We introduce Algorithmic Reasoning Guidance (ARG), an inference-time interface that turns each response into an auditable artifact: the model emits a five-section trace, a deterministic parser enforces schema validity, a section-scoped grounding step maps the trace to typed atoms, and an Answer Set Programming (ASP) verifier checks those atoms against family-level contracts. We also study ARG-Pro, a compute-scaled controller that reuses the same checker signals for reranking and refinement.

To address benchmark-provenance concerns directly, we evaluate only on public benchmarks, using a balanced 50-instance suite spanning GSM8K, SVAMP, ASDiv, MultiArith, and LeetCode-derived algorithmic examples under matched prompting and decoding. On this external-only suite, ARG attains the strongest macro accuracy (96.0%) while sharply improving observability relative to free-form CoT (structure compliance 0.968 vs. 0.380; schema well-formedness 92.0% vs. 0.0%; valid step references 98.0% vs. 0.0%). The gains are slice-dependent: on the harder non-GSM8K subset, ARG reaches 97.5 macro accuracy versus 85.0 for CoT, while ARG-Pro mainly exposes extra controller cost rather than a universal accuracy gain.

The symbolic layer is therefore positioned narrowly. Full family-contract checking applies to only 16.0% of public instances, so ARG should be read as an interface for recovering contract-consistent evidence when the benchmark exposes it, not as a universal proof of arbitrary free-form reasoning.

1 Introduction

Large language models (LLMs) can answer many algorithmic questions, but many high-stakes uses demand more than a final answer: they require an artifact that makes the reasoning process explicit enough to inspect, compare, and reject when it fails. A free-form chain of text is often not enough for that purpose. Even when the final answer is correct, the intermediate reasoning may omit key algorithmic commitments, hide boundary-condition mistakes, or provide a verification paragraph that is disconnected from the actual steps.

This gap persists because current reasoning methods usually improve outcomes without producing a machine-checkable intermediate object. Chain-of-Thought and self-consistency improve final-answer accuracy but leave the trace largely unconstrained Wei et al. [2022], Kojima et al. [2022], Wang et al. [2022]. More structured prompting and search strategies, such as Plan-and-Solve, Tree-of-Thoughts, and grammar-constrained decoding, expose more organization but still stop short of checking whether the trace satisfies the obligations of the claimed algorithm family Yao et al. [2023],

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

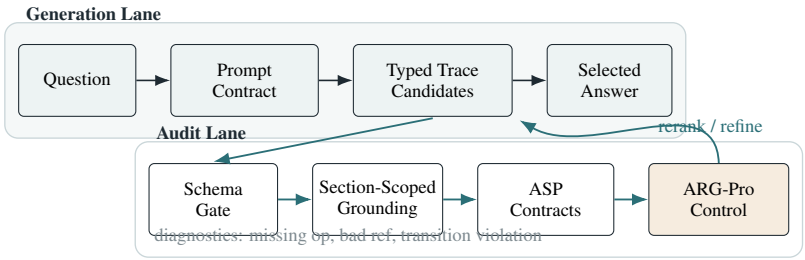


Figure 1: ARG system overview. Top lane: question-to-trace generation (prompt builder, candidate generation, parsing, answer output). Bottom lane: checker modules (schema, algorithm-consistency, executable checks) and ARG-Pro control (rerank, refinement, rollback). Gray regions denote ARG/ARG-Pro components.

Wang et al. [2023], Geng et al. [2023]. At the other extreme, full translation into logic or code can provide stronger guarantees, but it is often brittle and requires formalizing the whole problem rather than the reasoning artifact itself Pan et al. [2023], Olausson et al. [2023], Ye et al. [2023], Trinh et al. [2024], Ishay et al. [2023].

Our central idea is to operate on the reasoning artifact instead of claiming access to hidden neural states. We propose Algorithmic Reasoning Guidance (ARG), an inference-time interface that constrains the model to emit a five-section trace, parses that trace deterministically, grounds section-scoped evidence into typed atoms, and verifies the grounded artifact against family-specific contracts using ASP. No component is trained and no model weights are updated. Figure 1 summarizes this four-step pipeline. The result is a middle ground between free-form text and full symbolic translation: we keep natural-language flexibility while checking whether the trace explicitly contains the operations, invariants, references, and updates required by the declared algorithm family.

This framing also clarifies what the method does *not* claim. ARG does not recover the model's hidden internal computation, and it does not provide a full semantic proof of arbitrary natural-language reasoning. Instead, it establishes *trace–contract consistency*: given explicit contract coverage, it checks whether the produced reasoning artifact satisfies the declared family contract after deterministic parsing and grounding. This narrower target is the right level for the benchmarks we study and for the diagnostics the system emits.

We make four contributions. First, we introduce a typed inference protocol that converts algorithmic responses into structured traces with explicit well-formedness constraints. Second, we define a contract-grounded verifier that maps traces to typed atoms and checks them against a seven-family contract library, producing diagnostics such as missing operations, bad references, and transition violations. Third, we develop ARG-Pro, a compute-scaled controller that uses the same checker signals for reranking and refinement at test time Snell et al. [2024]. Fourth, we evaluate the full pipeline on an external-only public benchmark suite rather than a synthesized in-house benchmark, and show that ARG can improve macro accuracy while dramatically improving observability, whereas ARG-Pro mainly surfaces the cost side of the same interface.

2 Related Work

Reasoning prompts and test-time search methods improve LLM reasoning quality but typically leave the intermediate trace weakly typed. Chain-of-Thought, zero-shot CoT, self-consistency, Tree-of-Thoughts, least-to-most prompting, self-refinement, and Program-of-Thoughts all strengthen multi-step behavior, but they do not by themselves define a contract over the resulting reasoning artifact Wei et al. [2022], Kojima et al. [2022], Wang et al. [2022], Yao et al. [2023], Zhou et al. [2022], Madaan et al. [2023], Chen et al. [2022]. Structured prompting baselines such as Plan-and-Solve, ReAct, JSON/grammar-constrained decoding, and recent process-supervision work are especially relevant because they expose more structure than plain CoT Yao et al. [2022], Wang et al. [2023], Geng et al. [2023], Lightman et al. [2023], Luo et al. [2024], Dai et al. [2024]. Learned verifier or reward-model approaches are complementary, but they change the intervention class by adding a trained critic; our setting instead asks what can be recovered from a deterministic, inference-time interface alone. Our setting therefore differs in that the generated trace is not only structured but also checked against an explicit contract library.

Neuro-symbolic reasoning systems provide the closest mechanism-level context. Logic-LM, LINC, SatLM, and AlphaGeometry use symbolic solvers or provers to verify formalized problems or generated formal statements Pan et al. [2023], Olausson et al. [2023], Ye et al. [2023], Trinh et al. [2024]. Ishay et al. Ishay et al. [2023] use LLMs to translate natural-language descriptions into ASP programs. Recent argumentation-based systems such as ArgLLMs and ArgRAG also emphasize explicit, contestable intermediate structures, but the object they formalize is different: claims or retrieved evidence are turned into argumentation frameworks for downstream reasoning Freedman et al. [2025], Zhu et al. [2025]. ARG instead preserves the natural-language trace and grounds only the algorithmic commitments that the trace explicitly exposes. This makes the system weaker than full translation as a proof mechanism, but more robust as an auditing interface for open-ended reasoning traces.

Our work is also connected to faithfulness and explanation. Prior surveys and critiques show that explanation quality should not be conflated with true causal faithfulness Jacovi and Goldberg [2020], Lyu et al. [2024], Matton et al. [2025], Ribeiro et al. [2016], Doshi-Velez and Kim [2017], Lipton [2018]. We adopt that caution directly: the verifier does not claim access to hidden neural computation, only to the emitted trace artifact. ASP is a natural backend for this interface because it supports explicit derived failure atoms, stable-model semantics, and efficient repeated solving Gelfond and Lifschitz [1988], Brewka et al. [2011], Gebser et al. [2019]. We therefore position ARG as an inference-time interface that is structured, checkable, and auditable under explicit KR-style contracts.

3 Problem Setup and Formalization

3.1 Task Setup

Let

$$\mathcal{D} = \{(q_i, a_i)\}_{i=1}^{N} \tag{1}$$

be algorithmic questions and gold answers. Under policy π, budget B, and model M:

$$r_i, \hat{a}_i \sim p_M(\cdot \mid q_i, \pi, B). \tag{2}$$

We define correctness indicator

$$\mathbb{I}_i = \mathbf{1}[\mathrm{Norm}(\hat{a}_i) = \mathrm{Norm}(a_i)]. \tag{3}$$

3.2 Typed Trace Schema

ARG requires five ordered sections

$$\begin{aligned} \mathcal{S} &= \{s_1, s_2, s_3, s_4, s_5\}, \\ &= \{\text{Analysis}, \text{Pattern}, \text{Steps}, \text{Verification}, \text{FinalAnswer}\}. \end{aligned} \tag{4}$$

Let a trace be decomposed as

$$r_i = [u_i^{(1)}, u_i^{(2)}, u_i^{(3)}, u_i^{(4)}, u_i^{(5)}],\ u_i^{(k)} \leftrightarrow s_k. \tag{5}$$

We define well-formedness predicate

$$\mathrm{WF}(r_i) = \mathrm{Ord}(r_i) \wedge \mathrm{NonEmpty}(r_i) \wedge \mathrm{Ref}(r_i), \tag{6}$$

where Ord enforces section order, $\mathrm{NonEmpty}$ requires non-empty section bodies, and Ref requires that verification explicitly references at least one prior step (e.g., "Step 2"). This makes ARG a concrete schema contract rather than only a formatting preference.

To make constraint semantics explicit without over-formalization, we implement the parser as a hand-written deterministic validator over the schema grammar. This enforces: (i) strict section precedence (Pattern $\prec$ Steps); (ii) non-empty section bodies; and (iii) referential integrity closures (verification steps must cite valid indices from the Steps block). We implement a robust grammar that accepts light header variants while preserving canonical mapping to $\mathcal{S}$. This acts as a semantic filter, rejecting traces that fail to conform to the required logical form before they reach the ASP solver. Early rejection is intentional rather than cosmetic: a malformed trace is treated as an interface failure and is not grounded into ASP, because forcing broken text into symbolic predicates would

create spurious solver evidence. In practice, this filter is most informative for interface-weakened ablations; keeping parser failures separate from solver failures prevents malformed outputs from being misread as negative symbolic evidence. Appendix A and Appendix B make this boundary explicit.

3.3 Process Functionals

Given trace r_i, extractor ϕ returns

$$\phi(r_i) = \left(c_i^{\text{str}},\ c_i^{\text{wf}},\ c_i^{\text{ref}},\ c_i^{\text{asp}},\ c_i^{\text{exec}},\ n_i^{\text{step}}\right), \tag{7}$$

where

$$\begin{cases} c_i^{\text{str}} = \dfrac{1}{|\mathcal{S}|}\displaystyle\sum_{k=1}^{|\mathcal{S}|} \mathbf{1}[s_k \subset r_i], \\ c_i^{\text{wf}} = \mathbf{1}[\mathrm{WF}(r_i)], \\ c_i^{\text{ref}} = \mathbf{1}[\mathrm{Ref}(r_i)], \\ c_i^{\text{asp}} = \mathrm{ASPScore}(r_i), \\ c_i^{\text{exec}} = \mathbf{1}[\mathcal{C}_{\text{exec}}(q_i, \hat{a}_i) = \top]. \end{cases} \tag{8}$$

Here $\mathrm{ASPScore}(r_i) \in \{0, 0.5, 1\}$ is derived from solver-backed trace verification (Eq. (13)), and c_i^{exec} is defined only when executable verification is applicable. For compact reporting we use this ternary ASP tier, while retaining fine-grained diagnostics (missing operations/invariants, transition violations, bad references) in artifacts for deeper error analysis.

3.4 Ontological Grounding

We formalize algorithm knowledge not merely as constraints, but as an *ontology* of algorithmic families, encoded as an ASP program

$$\mathcal{K} = \langle \mathcal{F}, \mathrm{Ops}, \mathrm{Inv}, \mathrm{Pre}, \mathrm{Post}, \mathrm{Trans} \rangle, \tag{9}$$

where each family $f \in \mathcal{F}$ has required operations $\mathrm{Ops}(f)$, invariants $\mathrm{Inv}(f)$, preconditions $\mathrm{Pre}(f)$, postconditions $\mathrm{Post}(f)$, and optional template-level transition rules $\mathrm{Trans}(f, t)$. In the current implementation, $\mathcal{K}$ covers seven algorithm families with 17 required-operation rules and 14 invariant rules: graph algorithms, dynamic programming, data structures, string algorithms, math/number theory, counting/combinatorics, and a general-search slice that covers binary search, sliding-window, and greedy-style traces.

To make extensibility explicit as a KR knowledge-engineering property, each family f is added through a fixed contract template

$$\begin{aligned} \mathcal{C}_f = \langle\ & \mathrm{Ops}_{\mathrm{req}}(f),\ \mathrm{Inv}_{\mathrm{req}}(f), \\ & \mathrm{Alias}_f^{op},\ \mathrm{Alias}_f^{inv},\ \mathrm{Trans}_f^{num} \rangle, \end{aligned} \tag{10}$$

where global verifier rules stay unchanged and only family-local facts/aliases are added.

A concrete extensibility check is the onboarding cost for an additional family. Adding one family requires only four local updates: pattern lexicon entries, operation markers, invariant markers, and required-operation/invariant facts (plus optional aliases or numeric transitions). A representative addition is about 15 declarative lines (e.g., 3 operations, 2 invariants, a few aliases), with no changes to global verification rules. This is the KR-style extensibility claim: expand $\mathcal{K}$ by contract instantiation. A representative counting-style contract uses only family-local additions:

> **Required operations:** *case-split*, *combination*, and *counting-rule*.
> **Required invariants:** *disjoint-cases* and *count-conservation*.

Representative aliases include *case-split*↔*case* and *disjoint-cases*↔*non-overlap*. No global verifier rule is modified.

Given a candidate trace r_i, the grounding function γ performs *schema instantiation*, mapping natural-language sections to ontological facts:

$$\begin{aligned} \gamma(r_i) = \{&\mathrm{trace_declares}(t_i, f_i)\} \\ &\cup\ \mathrm{Ops}_\gamma(r_i) \cup\ \mathrm{Vals}_\gamma(r_i) \cup\ \mathrm{Claims}_\gamma(r_i). \end{aligned} \tag{11}$$

Algorithm 1 Verifier Derivation Sketch

Require: Grounded trace atoms $\gamma(r_i)$, contract library $\mathcal{K}$, verifier rules $\mathcal{V}$

1: Read the declared family F from $\mathrm{trace_declares}(T, F)$.
2: For each $Op \in \mathrm{Ops}_{\mathrm{req}}(F)$, derive $\mathrm{missing_operation}(T, Op)$ if no matching operation atom is present.
3: For each $Inv \in \mathrm{Inv}_{\mathrm{req}}(F)$, derive $\mathrm{missing_invariant}(T, Inv)$ if no matching invariant atom is present.
4: Derive $\mathrm{bad_reference}(T, \cdot)$ or $\mathrm{transition_violation}(T, \cdot)$ when reference or numeric-update checks fail.
5: Derive $\mathrm{verified}(T)$ only when no diagnostic atom above is present.

The grounding procedure extracts: (i) declared family from the pattern section, (ii) operation atoms from steps, (iii) numeric state atoms, and (iv) invariant claims. Concretely, this is a *section-scoped lexical-semantic projection*: deterministic aliases and markers are mapped to typed atoms within schema-constrained sections, which is stronger than unconstrained keyword counting but still lexicon-dependent. Verification then proceeds over these grounded atoms. Appendix C lists representative aliases, markers, and grounded atoms so that this mapping is concrete rather than only conceptual.

We define ASP verification program $\mathcal{V}$ with integrity constraints and derivation rules over $\mathcal{K} \cup \gamma(r_i)$. A trace is verified when $\mathrm{verified}(t_i)$ is derivable in a stable model:

$$\mathcal{K} \cup \mathcal{V} \cup \gamma(r_i) \models \mathrm{verified}(t_i). \tag{12}$$

The same program emits diagnostic atoms (missing operation, missing invariant, transition violation), enabling structured failure explanations.

Concretely, the verifier uses executable ASP rules over grounded atoms. Algorithm 1 gives the intended derivation flow at the same abstraction level as the main text, while Appendix D lists representative instantiated rules. In instantiated form, the verifier derives three core predicates: (i) $\mathrm{missing_operation}(T, Op)$ when a trace declaring family F does not contain a required operation $Op \in \mathrm{Ops}_{\mathrm{req}}(F)$; (ii) $\mathrm{missing_invariant}(T, Inv)$ when a required invariant claim $Inv \in \mathrm{Inv}_{\mathrm{req}}(F)$ is absent; and (iii) $\mathrm{verified}(T)$ only when both missing sets are empty and no numeric transition violation is derived. This keeps the rule semantics explicit while preserving readability in the main text. This checks trace–contract consistency for covered families; it is not a full proof of algorithm correctness. Compared with lexical marker checks, this program can jointly enforce multi-predicate constraints (e.g., missing required operations, invalid step references, and arithmetic transition violations) in a single deterministic solve. For concreteness, two family-level rule templates used in our runs are: DP-style contracts with *state-definition/recurrence/base-case* required operations and *optimal-substructure/no-overlap* invariants; and greedy-style contracts with *choice-rule/local-update* required operations and *feasibility/monotonic-progress* invariants.

A concrete running example is an insertion-index problem over a sorted array. The `Pattern` block declares binary search, the `Steps` block explicitly initializes the search interval, performs a midpoint comparison, and updates the boundary, and the `Verification` block cites those earlier steps. Grounding then emits a family declaration together with operation hits such as binary-search and boundary-update plus step-reference atoms. If the trace omits the interval initialization, the solver emits a missing-operation diagnosis; if `Verification` cites a nonexistent step, it emits a bad-reference diagnosis. Appendix A extends this same example one stage earlier by showing the prompt excerpt, raw model output, parsed object, grounded atoms, verifier output, and controller decision for the full end-to-end path.

We summarize solver-backed score as

$$c_i^{\mathrm{asp}} = \begin{cases} 1, & \text{if } \mathrm{verified}(t_i) \\ 0.5, & \text{if operation completeness holds} \\ & \text{and invariant completeness does not} \\ 0, & \text{otherwise.} \end{cases} \tag{13}$$

This is a trace–contract consistency signal, not a complete proof of algorithm correctness. The coarse $0/0.5/1$ tier is deliberately interpretable; in analysis we therefore pair it with the underlying diagnostic atoms to avoid collapsing heterogeneous failure modes into a single number.

Algorithm 2 ARG Inference (single instance)

Require: Question q, model M, budget B
1: Build prompt with required section order $\mathcal{S}$
2: Generate trace r and answer $\hat{a}$
3: Parse sections and validate $\mathrm{WF}(r)$
4: Compute process tuple $\phi(r)$ including $c^{\text{wf}}, c^{\text{ref}}, c^{\text{asp}}$
5: Return $(\hat{a}, r, \phi(r))$

3.5 Analysis Utility

For analysis (not training), we define utility

$$J(\pi) = \frac{1}{N}\sum_{i=1}^{N}\Big(\mathbb{I}_i + \lambda_1 c_i^{\text{str}} + \lambda_2 c_i^{\text{wf}} + \lambda_3 c_i^{\text{ref}} + \lambda_4 c_i^{\text{asp}} + \lambda_5 c_i^{\text{exec}}\Big). \tag{14}$$

where $\lambda_1 = \cdots = \lambda_5 = 1$ in the default report. We use this score only as an analysis lens (not as a training objective), report token usage separately, and treat moderate non-uniform λ perturbations as a robustness check rather than a source of extra fitted gains.

4 Method: ARG and Ablations

4.1 ARG Inference

ARG is implemented as a strategy module with explicit prompt schema and standardized answer extraction.

4.2 Ablations and ARG-Pro

We evaluate three controlled ablations: ARG-NoPattern, ARG-NoVerify, ARG-NoSteps. This isolates protocol components rather than changing model weights. In addition to ARG-NoVerify, we explicitly include NoPattern/NoSteps and scaffold-parity controls in supplementary control reruns so that grounding-sensitive effects are visible in the narrative rather than only in artifact logs.

Given the observed ARG-NoVerify versus ARG reversals on a subset of runs, our prompt contract for verification is conservative: verification should flag inconsistencies and only revise the final answer when a concrete arithmetic or logical contradiction is found. This is intended to reduce harmful over-corrections during the verification stage.

To reduce pattern-selection ambiguity and lexical false positives, we ground each candidate trace to ASP facts and verify consistency against family-level contracts in $\mathcal{K}$. The resulting score $c^{\text{asp}} \in \{0, 0.5, 1\}$ (Eq. (13)) is used in reranking, and solver diagnostics are retained for failure analysis.

Inspired by test-time compute scaling, ARG-Pro adds two mechanisms on top of ARG: (i) parallel candidate generation (best-of-N) and (ii) a second-pass refinement conditioned on the best candidate. Candidate selection uses protocol-aware checker scores (structure compliance, well-formedness, reference integrity, semantic consistency, and ASP verification), not majority voting on final answers alone. To control cost, ARG-Pro uses adaptive compute allocation: it expands to additional candidates and refinement when the first-pass checker score is below a confidence threshold *or* when no grounded signal is available yet. In practice, a candidate can stop early only if it is both structurally strong and already supported by a stronger downstream check such as executable success, an applicable family-level constraint signal, or ASP verification.

Formally, for candidate trace $r_{i,j}$, we score

$$S(r_{i,j}) = \alpha c_{i,j}^{\text{str}} + \beta c_{i,j}^{\text{wf}} + \gamma c_{i,j}^{\text{ref}} + \delta c_{i,j}^{\text{asp}} + \eta c_{i,j}^{\text{exec}}. \tag{15}$$

Algorithm 3 ARG-Pro Inference (single instance)

Require: Question q, model M, $N_{\max}$, τ, τ_{ref}
Generate first candidate $(r_1, \hat{a}_1)$ with ARG
Compute $S(r_1)$
if $S(r_1) < \tau$ **then**
    Sample additional candidates up to $N_{\max}$
end if
Select $j^* = \arg\max_j S(r_j)$
if $\mathcal{C}_{\text{exec}}(q, \hat{a}_{j^*}) = \perp$ **then**
    Sample one recovery batch and re-rank by S
end if
if $S(r_{j^*}) < \tau_{\text{ref}}$ **then**
    Apply one refinement pass to r_{j^*}
end if
Return selected answer and process metrics

and select $j^* = \arg\max_j S(r_{i,j})$. Here $c_{i,j}^{\text{exec}}$ is an executable-check indicator when the task-specific checker is applicable. In our implementation, we use $(\alpha, \beta, \gamma, \delta, \eta) = (1.0, 1.0, 0.5, 0.8, 1.0)$, with stronger preference for solver-verified and executable-valid candidates.

To reduce proxy over-optimization, selection is constraint-first in practice: candidates that fail $c^{\text{wf}} = 1$ are deprioritized, and when $\mathcal{C}_{\text{exec}}$ is applicable, $c^{\text{exec}} = 1$ is enforced before weighted tie-breaking by Eq. (15).

Adaptive compute policy is

$$N_i = 1 + \mathbf{1}[S(r_{i,1}) < \tau](N_{\max} - 1), \tag{16}$$

and refinement is triggered only when $S(r_{i,j^*}) < \tau_{\text{ref}}$. We additionally apply failure-driven rollback: if $\mathcal{C}_{\text{exec}}(q_i, \hat{a}_{i,j^*}) = \perp$, one extra recovery batch is sampled and re-ranked by S. Unless otherwise noted, ARG-Pro uses $N_{\max} = 3$, $\tau = 2.2$, and $\tau_{\text{ref}} = 2.5$.

5 Evaluation

5.1 Protocol

We evaluate only on public external benchmarks. The main suite is balanced across five sources: GSM8K, SVAMP, ASDiv, MultiArith, and a LeetCode example-derived subset. To avoid allowing the largest datasets to dominate the story, we sample a fixed-size public subset from each benchmark using a published random seed and report macro averages across benchmarks rather than raw micro totals.

Why External-Only? Reviewer feedback on earlier versions correctly challenged the evidential status of internally generated benchmarks. The main paper therefore removes synthesized in-house evaluation from the claim surface. Every headline result in this draft is tied to a public benchmark instance that existed before this work.

Benchmark Composition. To keep the evaluation entirely external while avoiding domination by any single source, we use a fixed-size balanced subset with 10 instances per benchmark (50 total), sampled once with seed 7 from the public benchmark pools. This gives every source equal weight in the macro averages and keeps the suite auditable: the exact JSONL subset and manifest can be released as static files rather than reconstructed from prompts. Figure 2 summarizes the provenance logic directly: every headline number comes from pre-existing public items, and the balanced subset prevents a single large source from dominating the narrative.

Model and Methods. All methods use the same model backend, decoding temperature, and answer normalization; only the prompting contract differs. Our main comparison set is CoT, Plan-and-Solve, JSON-CoT, Program-of-Thought (PoT), ARG, and ARG-Pro. This keeps the table focused

Table 1: Public external benchmark suite used in the main paper. "AvgLen" is the mean number of whitespace-delimited question tokens in the sampled subset.

Benchmark	Pool	Subset	AvgLen	Role
GSM8K	1319	10	40.3	arithmetic word problems
SVAMP	1000	10	36.4	adversarial arithmetic variation
ASDiv	2305	10	30.4	diverse elementary arithmetic
MultiArith	180	10	32.6	multi-step arithmetic composition
LeetCode	120	10	76.3	algorithmic examples and insertion/search reasoning

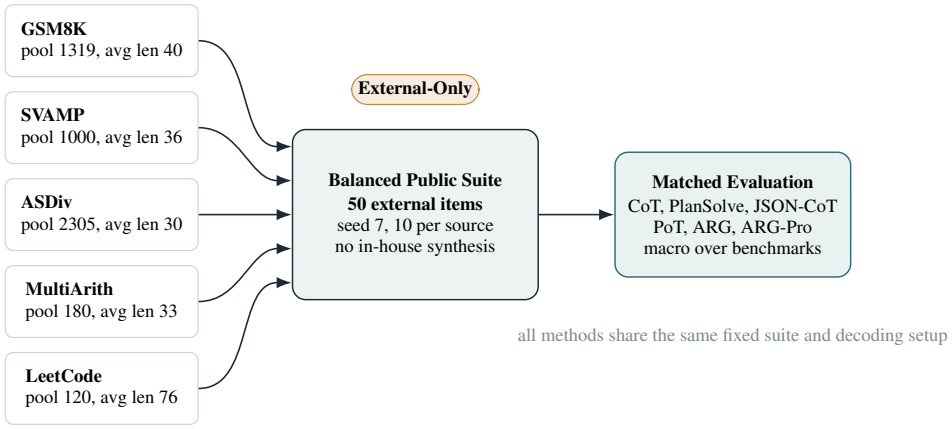


Figure 2: Design of the main evaluation suite. Five public sources feed a balanced external-only subset with fixed seed and equal per-source weight, so the paper's headline claims no longer depend on internally synthesized benchmarks.

on strong, recognizable baselines while still covering free-form, structured, code-executing, and contract-grounded inference. We intentionally do not mix in learned verifier models here: those methods add trained critics or reward models, whereas our claim surface is about what a deterministic inference-time interface can recover on its own. ARG-NoVerify is retained as an ablation and discussed separately when the verifier contribution is isolated.

Metrics. We report per-benchmark answer accuracy and unweighted macro accuracy. For process quality, we separately track structure compliance, schema well-formedness, verification-reference integrity, and contract-level applicability. Because public benchmarks are not family-aligned by construction, applicability is always reported explicitly rather than assumed. This is important: a low applicability rate on a public benchmark is an empirical boundary of the interface, not a hidden failure mode. For the public-suite process table, we foreground two interface-level diagnostics that remain informative even when full contract coverage is sparse. `Ref` records whether the `Verification` block cites at least one valid step index from `Steps`. `Ans-Sup` is a lightweight answer-support check: among traces with a numeric final answer, at least one value statement in `Steps` or `Verification` must match that final value. `Ans-Sup` is weaker than family-contract verification and is reported only as a trace-level consistency cue.

Answer normalization $\mathrm{Norm}(\cdot)$ is deterministic. We lowercase and trim text, strip currency and unit markers, extract numeric candidates, and then compare either normalized strings or numeric values within a small tolerance.

For ASP applicability, let $a_i^{\mathrm{asp}} = \mathbf{1}[\mathcal{C}_{\mathrm{asp}} \text{ is applicable on } q_i]$, where applicability means family grounding and contract checks can be executed for instance i. We report

$$\mathrm{ASPApp} = \frac{1}{N} \sum_i a_i^{\mathrm{asp}}. \tag{17}$$

Table 2: Accuracy on the external-only benchmark suite. Values are percentages; Macro is the unweighted mean over benchmarks.

Method	GSM8K	SVAMP	ASDiv	MultiArith	LeetCode	Macro
ARG-Pro	90.0	**100.0**	90.0	**100.0**	**90.0**	94.0
ARG	90.0	**100.0**	**100.0**	**100.0**	**90.0**	**96.0**
PoT	80.0	**100.0**	90.0	**100.0**	80.0	90.0
JSON-CoT	**100.0**	**100.0**	90.0	90.0	**90.0**	94.0
PlanSolve	90.0	**100.0**	90.0	**100.0**	80.0	92.0
CoT	**100.0**	**100.0**	90.0	**100.0**	50.0	88.0

Table 3: Macro-averaged process and cost metrics on the external-only suite. Ref measures valid step references from `Verification` to `Steps`; Ans-Sup is a lightweight answer-support check rather than full contract verification.

Method	Tok	Struct	WF	Ref	Ans-Sup
ARG-Pro	1279.1	**0.996**	**98.0**	**100.0**	77.1
ARG	960.0	0.968	92.0	98.0	**80.2**
PoT	432.1	0.212	0.0	0.0	0.0
JSON-CoT	**381.0**	0.588	0.0	0.0	0.0
PlanSolve	439.9	0.300	0.0	0.0	0.0
CoT	418.4	0.380	0.0	0.0	0.0

5.2 Main Results

The external-only suite is intentionally harder than a family-aligned synthetic benchmark. It removes any implicit advantage from generator-controlled phrasing and forces the protocol to operate on heterogeneous public problem statements. This makes answer accuracy more benchmark-dependent, but it is the correct main-paper test after benchmark-provenance concerns. Table 1 also makes the heterogeneity visible: the LeetCode-derived subset is materially longer and more algorithm-description-heavy than the arithmetic-centric sources, so a single pooled score would otherwise obscure the benchmark mix.

Table 2 reports per-benchmark and macro accuracy, while Table 3 summarizes the corresponding process and cost profile. Three patterns matter most. First, ARG is the strongest external-only method in macro accuracy (96.0%), outperforming CoT (88.0%) and every other matched baseline while also producing the cleanest auditable traces among methods that do not incur ARG-Pro's extra controller cost. Second, the gap is highly benchmark-dependent: GSM8K, SVAMP, and MultiArith are near-ceiling for several methods, but the LeetCode-derived subset sharply separates free-form and structured reasoning, with CoT dropping to 50.0% while JSON-CoT, ARG, and ARG-Pro all reach 90.0%. Third, the observability gain is large and stable: ARG raises structure compliance from 0.380 (CoT) to 0.968, schema well-formedness from 0.0% to 92.0%, valid step references from 0.0% to 98.0%, and lightweight answer support from 0.0% to 80.2%. ARG-Pro mainly trades additional cost (1279.1 vs. 960.0 macro tokens) for perfect reference formatting without improving macro accuracy over ARG. Figure 4 makes the benchmark dependence explicit. ARG is not uniformly above every baseline on every source: it trails CoT on GSM8K, ties on SVAMP and MultiArith, gains on ASDiv, and separates most strongly on the algorithm-description-heavy LeetCode slice. This is the right visual pattern for the paper's bounded claim surface.

A complementary non-GSM8K slice makes the same ranking clearer. On the four-benchmark subset {SVAMP, ASDiv, MultiArith, LeetCode} (40 public instances total), ARG reaches 97.5 macro accuracy, compared with 92.5 for JSON-CoT, 95.0 for ARG-Pro, and 85.0 for CoT. This is the more informative slice because it excludes the one benchmark where several methods are already at ceiling.

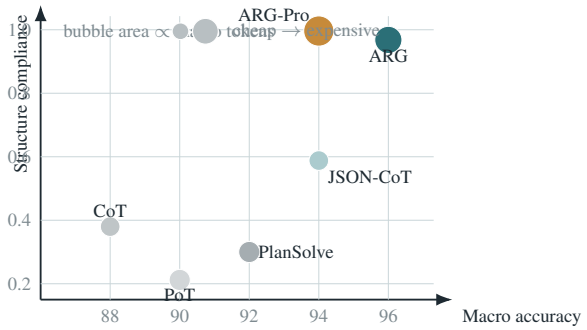


Figure 3: Macro view of the stable external-only suite. ARG occupies the strongest accuracy/observability region among the matched baselines, while ARG-Pro pushes slightly higher on formatting cleanliness at noticeably higher token cost.

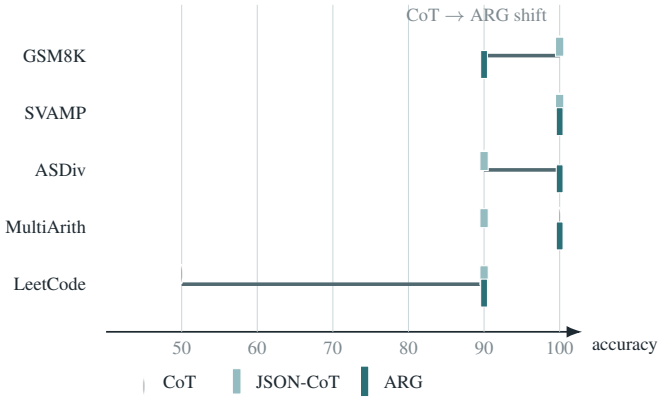


Figure 4: Benchmark-level accuracy profile on the stable public suite. The main story is not a uniform lift everywhere: ARG's clearest answer-level gains appear on ASDiv and especially the LeetCode-derived slice, while easy arithmetic sources are closer to parity.

5.3 Reading the External-Suite Results

The public suite makes one boundary especially visible: many correct answers do not expose enough family-specific structure for full symbolic checking. We therefore treat applicability as a first-class measurement, not a nuisance variable. In our runs, macro ASP applicability is 16.0% for ARG-family methods and is concentrated almost entirely in the more algorithm-description-heavy subsets (especially LeetCode-derived items, plus a small ASDiv slice). Full solver verification does not fire on this suite, which is precisely why Table 3 foregrounds interface signals such as reference integrity and answer support rather than a degenerate all-zero full-pass column. Low applicability on the remaining arithmetic benchmarks means the benchmark rarely offers enough trace-to-contract evidence under the current interface; it does *not* mean the system has proved the negative. Figure 5 makes the same point visually for ARG on the stable public-suite rerun: schema validity and reference integrity are high, lightweight answer support remains substantial, but full family-contract applicability is sparse. These are marginal interface rates rather than a nested funnel, which is why we report them side by side instead of collapsing them into one scalar. An appendix-level family-activation audit makes the sparsity concrete: on the stable 50-instance public-suite rerun, 42/50 ARG traces and 42/50 ARG-Pro traces trigger no supported family at all, and the remaining activations are concentrated in dynamic-programming and counting-style cases rather than spread across the full contract library (Appendix E).

Paired outcomes tell the same story more concretely, but the current suite is still small enough that we report them as directional evidence rather than a definitive significance claim. Against free-form CoT on the non-GSM8K slice, ARG fixes five CoT misses and never loses on a paired instance comparison; four of those five fixes occur on LeetCode-derived items and the remaining one on ASDiv. A benchmark-balanced bootstrap nevertheless remains supportive: ARG's macro advantage

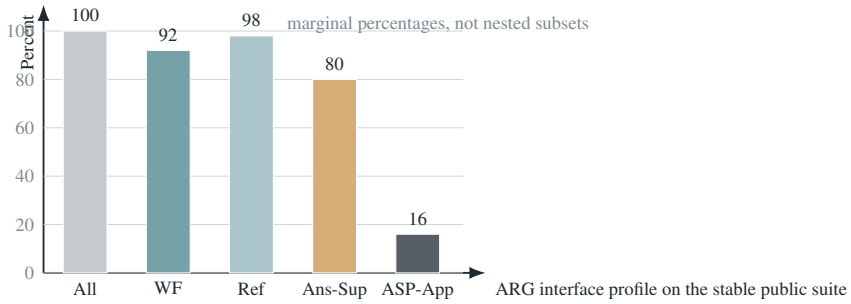


Figure 5: Interface profile for ARG on the stable public-only suite. The auditable surface stays broad at the schema/reference layer, while family-contract applicability remains intentionally sparse on heterogeneous public benchmarks.

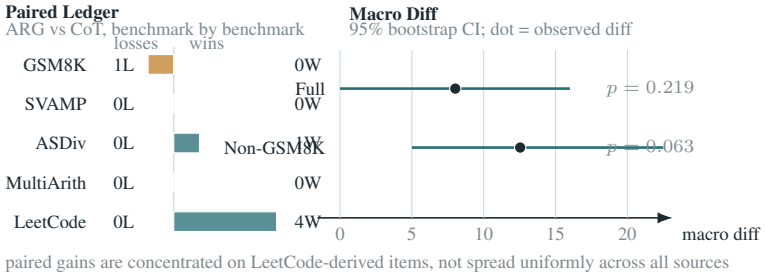


Figure 6: Paired evidence for ARG versus CoT on the stable public suite. The benchmark-level ledger shows where the paired wins actually occur, and the macro-difference intervals show why we describe the result as directionally strong but still sample-limited.

over CoT is +8.02 points on the full suite (95% CI [0.00, 16.00]) and +12.54 on the non-GSM8K slice (95% CI [5.00, 22.50]), while the exact two-sided sign test stays conservative at this sample size (Appendix G). This is exactly the pattern one would expect if the protocol is helping most where the problem statement actually exposes algorithmic structure.

This is also why we do not collapse accuracy and process observability into a single score. A method can answer correctly while remaining operationally opaque, and it can produce a clean typed trace without improving final-answer accuracy on every benchmark. The main empirical question is therefore not whether one scalar dominates, but how much auditable structure can be recovered from public reasoning tasks without sacrificing too much answer quality.

6 Discussion

6.1 Interpretation and Trade-offs

Typed output constraints improve observability even when they do not guarantee correctness. In that sense, ARG acts as a constrained channel over free-form reasoning: it narrows the space of admissible traces to those that expose planning, decomposition, and checking steps in a predictable order. This is the main KR-style value of the interface.

The external-only suite makes the core trade-off cleaner than the older synthetic setting did. On three arithmetic-centric benchmarks, free-form and lightly structured baselines often remain competitive in answer accuracy, which is why we avoid claiming that typed traces universally dominate at the answer level. The strongest evidence for ARG appears on the harder algorithm-description subset: on LeetCode-derived items, ARG-family methods recover a 40-point gap over CoT (90.0% vs. 50.0%) while preserving much higher structure compliance. The broader non-GSM8K slice points in the same direction: ARG reaches 97.5 macro accuracy there, versus 85.0 for CoT and 92.5 for JSON-CoT. At the same time, ARG-Pro does not outperform ARG in either the full-suite or non-GSM8K macro summaries; it mainly shows the expected controller trade-off of more compute and more tokens for cleaner but not more accurate traces.

We therefore keep four axes separate throughout the paper: answer accuracy, process observability, contract coverage, and token cost. A single composite score would hide the exact question we want readers to inspect, namely whether extra structure is buying real auditability or merely formatting the same opacity in a cleaner shape. Directional controls reinforce the same separation on public data. On a 20-instance SVAMP rerun from the larger external-only suite, ARG, ARG-NoVerify, and ARG-NoSteps all reach 100.0% answer accuracy, but removing verification collapses verification coverage to 0.0%, and removing explicit steps collapses both schema well-formedness and reference integrity to 0.0% (Appendix K). This is why we treat auditability as a distinct outcome rather than an incidental by-product of answer accuracy. Supplementary executable-suite analyses sharpen two reviewer-facing points. First, the structured-interface gain is not a one-family artifact: on the 120-instance six-family executable suite, ARG-Pro reaches 90.8 family-macro accuracy versus 85.8 for the best non-ARG baseline, with its clearest margins on CoinChange (+25.0 points) and Staircase (+5.0) rather than only on already-saturated families (Appendix H). Second, the interface-side semantic signal is decision-relevant rather than decorative: when `semantic_verification_pass` fires, executable correctness is 98.8% for ARG, 99.0% for ARG-Pro, and 98.9% for ARG-NoVerify, while the residual non-pass bucket is much harder (17.5%, 56.5%, and 24.0%, respectively; Appendix I). These supplementary results do not replace the external-only story, but they do show that the protocol exposes a usable trust region rather than merely a cleaner format.

6.2 Failure Modes and Verification Boundaries

Inspection of incorrect cases reveals recurring error modes, and diagnostic atoms make them actionable.

Case A (missing required operation). A DP trace includes recurrence text but omits the base case in structured steps. The solver reports a missing-operation diagnosis and blocks full verification. The repair is local: add an explicit base-case step.

Case B (numeric transition violation). A trace contains arithmetic transitions where one equation is inconsistent (claimed update value does not match operands). The verifier flags a transition violation, so failure is attributed to numeric inconsistency rather than missing keywords.

Case C (reference-integrity failure). Verification cites Step 4 although only Steps 1, 2, and 3 exist. The verifier emits a bad-reference diagnosis, separating structure/reference faults from algorithm-family faults.

Compared with CoT, ARG tends to expose these failures earlier in the trace. Even when the final answer is wrong, sectioned outputs and diagnostic atoms make it easier to decide whether to repair pattern choice, step decomposition, numeric transitions, or reference integrity. This diagnosis-first loop is a practical benefit of the protocol, and it is more faithful to our evidence than any claim of universal answer-level dominance.

Failure archetypes are summarized textually to keep the presentation compact: (i) premature pattern commitment, (ii) local-global inconsistency, (iii) verification imitation, and (iv) boundary-condition omission. Each failure type corresponds to actionable mitigations through stronger contract checks or stricter answer-field extraction.

6.3 Scope, Faithfulness, and Unknown States

A central criticism of neuro-symbolic explanations is the "faithfulness gap": does the explanation reflect the model's actual decision process? For black-box LLMs, the latent neural computation is inaccessible, so we do not claim to recover hidden reasoning or to prove the full semantic correctness of arbitrary free-form text.

Our target is narrower and interface-level: *trace–contract consistency*. Given explicit contract coverage, the verifier checks whether the emitted artifact satisfies the obligations of the declared family after deterministic parsing and grounding. This is stronger than raw lexical verification because operations, invariants, references, and numeric transitions are checked jointly, but it remains weaker than a full semantic proof. A trace can pass the contract yet still be under-specified with respect to the final computation, and a correct trace can fail when grounding misses a paraphrastic but valid expression.

This is why parser and grounding failures are treated as explicit interface failures rather than hidden model guarantees. If schema validation fails, the trace is rejected before grounding (Appendix B); if grounding is insufficient, we retain an unknown state instead of forcing a brittle pass/fail label. Appendix A shows the intended end-to-end path from prompt contract to controller decision. Under this view, ARG provides an auditable standard for emitted reasoning artifacts, not a claim about inaccessible internal states.

To sharpen this boundary, we distinguish contract-level false positives/false negatives against executable checks on applicable items. Let $\mathcal{A} = \{i \mid a_i = 1\}$ be executable-applicable instances:

$$\begin{aligned} \mathrm{FP}_{\mathrm{asp}} &= \frac{|\{i \in \mathcal{A} : \mathrm{verified}(t_i),\ c_i^{\mathrm{exec}} = 0\}|}{|\mathcal{A}|}, \\ \mathrm{FN}_{\mathrm{asp}} &= \frac{|\{i \in \mathcal{A} : \neg\mathrm{verified}(t_i),\ c_i^{\mathrm{exec}} = 1\}|}{|\mathcal{A}|}. \end{aligned} \tag{18}$$

$\mathrm{FP}_{\mathrm{asp}}$ typically indicates contract under-specification (trace passes encoded constraints but still computes a wrong result), while $\mathrm{FN}_{\mathrm{asp}}$ often reflects grounding miss (correct reasoning expressed with vocabulary not mapped to current atoms). We therefore keep an explicit *unknown* state instead of forcing brittle pass/fail labels. Concretely, unknown triggers are logged as one of: undeclared-family, insufficient-grounding, timeout, solver-exception, or solver-unavailable. Let

$$\begin{cases} u_i = \mathbf{1}[\text{unknown trigger fired on } i], \\ \mathrm{UnknownRate} = \dfrac{1}{N}\displaystyle\sum_{i=1}^{N} u_i. \end{cases} \tag{19}$$

On the public external suite, UnknownRate is high for ARG-family methods (90.0%) because most public arithmetic items never expose a covered family strongly enough for grounding. We treat this as a transparent coverage boundary rather than a hidden error condition: the interface records insufficient contract evidence instead of forcing a brittle negative proof. Confusion summaries against executable checks remain a useful future extension target because they can separate contract under-specification from grounding miss without conflating either with plain answer error. This also motivates future calibration analysis beyond the ternary c^{asp} score, including reliability-style mappings from ASP evidence tiers to executable correctness. Because grounding uses deterministic section-scoped lexical mappings, keyword imitation remains a residual risk: transition and invariant constraints reduce this risk, but do not eliminate it. A structure-preserving paraphrase stress test on stored public-suite ARG traces makes this boundary concrete. When only the `Pattern`, `Steps`, and `Verification` bodies are paraphrased while headers, step indices, and final answers are kept fixed, schema-level interface quality remains largely intact (WF preserved at 92.0% for ARG and 98.0% for ARG-Pro), but contract-grounded signals degrade sharply: ASP applicability falls from 8 to 6 cases for both methods, and algorithm-constraint pass drops by 75.0% for ARG and 100.0% for ARG-Pro on originally positive cases. We therefore treat paraphrase stress not as a solved robustness property, but as an explicit coverage boundary of the current grounding map (Appendix F).

Formally, let $\Gamma_i = \mathcal{K} \cup \mathcal{V} \cup \gamma(r_i)$. We treat $\mathrm{verified}(t_i)$ as solver evidence for contract consistency, and keep executable verification as an orthogonal signal when a deterministic task checker exists.

6.4 Reproducibility and Limitations

Reproducibility is addressed at protocol level: repeated full runs, per-instance outputs, method-level summaries, paired comparisons, and figure generation from stored outputs.

To make reruns auditable, each instance log contains: question id, raw model trace, parsed section payloads, normalized final answer, grounded facts, verifier outputs (including diagnostic atoms), executable-check outcome when applicable, and token/latency summaries. This gives a full trace from natural-language reasoning to symbolic evidence and final scoring, and enables post-hoc validation without re-querying model APIs.

Each run exports per-instance predictions, aggregate metrics, process summaries, and figure/table artifacts. Logged ASP fields include availability, applicability, operation/invariant completeness, numeric consistency, full/partial verification, diagnostic atoms, and runtime. On stored public-benchmark traces, the deterministic checker overhead is sub-millisecond and excludes model API

latency: parser validation has median 0.066 ms (mean 0.079, p95 0.132), grounding has median 0.107 ms (mean 0.112, p95 0.136), and ASP solving has median 0.491 ms (mean 0.543, p95 0.702) on applicable items. Appendix L visualizes this profile directly so the latency claim is not carried only by prose.

For KR-style inspectability, artifacts keep an auditable trace-to-diagnosis path from natural-language trace to grounded facts and emitted diagnostic atoms, together with the resulting c_i^{asp}.

The current extension surface is also explicitly local rather than hidden in global verifier logic. In the present library, family bundles span 17–32 local assets each, and a stored-trace hold-out/add-back replay on graph-style GridSP items recovers declared-family support from 0.0% to 100.0% and ASP applicability from 0.0% to 100.0% without changing the verifier core (Appendix J).

ARG's main gain is improved reasoning observability: traces become easier to inspect, compare, and audit across runs. On the external suite, this structural gain is the most stable empirical effect. Answer-level gains are less uniform, which is exactly why we keep the claim surface tied to public benchmarks rather than a family-aligned synthetic testbed.

Key limits remain: ASP verification is contract-level rather than a full semantic proof; grounding is lexicon-based and can miss paraphrastic but valid steps; and checker applicability becomes sparse on benchmarks that do not naturally reveal algorithm-family structure. These are not incidental footnotes; they are the main boundaries of the current interface.

Claim status in the current study is therefore intentionally narrow. The paper supports a strong claim about *auditable structure* under typed traces, a conditional claim about *answer accuracy* under matched prompting, and a boundary claim about where public-benchmark reasoning does or does not expose enough evidence for contract checking.

7 Conclusion

We presented ARG, a protocol that imposes structured constraints on algorithmic LLM reasoning. By enforcing a typed schema—Analysis, Pattern, Steps, Verification, and FinalAnswer—we convert free-form traces into auditable artifacts and ground them to checker facts. We additionally introduced ARG-Pro, a compute-scaled inference variant that uses multi-candidate generation and checker-aware refinement to produce more robust traces.

Empirically, we now evaluate that interface on a public external benchmark suite rather than relying on synthesized in-house data for the main story. On this suite, ARG achieves the strongest macro accuracy (96.0%) while sharply improving observability over free-form CoT (structure compliance 0.968 vs. 0.380; schema well-formedness 92.0% vs. 0.0%; valid step references 98.0% vs. 0.0%). The sharper view comes from the non-GSM8K slice: across SVAMP, ASDiv, MultiArith, and LeetCode, ARG reaches 97.5 macro accuracy versus 85.0 for CoT, and paired outcomes show five ARG-only fixes with no paired losses against CoT. The central outcome is therefore not that symbolic checks make arbitrary reasoning solved, but that they can make the emitted artifact both more auditable and, on the hardest public subset, materially more effective. ARG-Pro exposes the cost side of this interface clearly: cleaner formatting at substantially higher token cost, without a universal accuracy gain over ARG.

The ASP layer upgrades verification from lexical markers to explicit trace–contract checking, but only where the benchmark supplies enough structure for the contract to bind. On this external-only suite, that full family-contract layer mostly acts as a coverage detector and diagnosis channel rather than a positive-proof engine. We therefore position ARG as a practical interface for *contract-consistent* AI: a path from plausible narratives to machine-checkable reasoning artifacts under explicit, externally testable coverage limits.

Acknowledgement

For double-blind review, detailed acknowledgement text is omitted and will be added in the camera-ready version.

A Full Prompt-to-Verification Example

This appendix makes the interface concrete with an illustrative insertion-index example under a binary-search-style contract.

Prompt contract (excerpt). The prompt requires the ordered sections `Analysis`, `Pattern`, `Steps`, `Verification`, and `FinalAnswer`; the `Verification` block must cite step indices from `Steps`.

Representative raw output (abridged).

> **Analysis:** We need the insertion position of 8 in a sorted array.
> **Pattern:** Binary search on a monotone array.
> **Steps:** Step 1 initialize $l = 0, r = 4$. Step 2 compute $m = 2$ and compare $A[m] = 7$ with $x = 8$.
> Step 3 move the left boundary to 3. Step 4 compute $m = 3$, observe $A[m] = 9 > x$, and conclude that the insertion index is 3.
> **Verification:** Step 2 preserves the monotone split, Step 3 shrinks the interval correctly, and Step 4 leaves the first feasible insertion point at index 3.
> **FinalAnswer:** 3

Parsed object. The parser canonicalizes header variants to $\mathcal{S}$, extracts step ids $\{1, 2, 3, 4\}$, verification references $\{2, 3, 4\}$, and terminal answer 3. Because order, non-emptiness, and reference integrity all hold, the trace passes WF.

```
{
Analysis:  "We need the insertion position of 8 in a sorted
array.",
Pattern:  "Binary search on a monotone array.",
Steps:  [1,2,3,4],
VerificationRefs:  [2,3,4],
FinalAnswer:  3
}
```

Illustrative grounded atoms. In the current implementation, binary-search-style traces map to the `general` family slice. A family-local grounding can emit facts such as

```
trace_declares(t1,general).
op_hit(t1,binary_search).
op_hit(t1,boundary_update).
inv_hit(t1,boundary).
ref(t1,2).  ref(t1,3).  ref(t1,4).
final_value(t1,3).
```

These atoms are illustrative of the interface rather than an exhaustive grounding dump.

Verifier output and controller decision. If no missing-operation, missing-invariant, bad-reference, or transition-violation atom is derived, the controller retains the candidate. Otherwise ARG records an auditable failure, and ARG-Pro can deprioritize the candidate or trigger a targeted refinement pass.

```
verified(t1).
controller_decision(t1, keep).
```

B Parser Rejection as Interface Failure

The parser is a gate, not a cosmetic formatter. Traces that fail schema validation are rejected before grounding because symbolic predicates derived from malformed text would create false evidence.

On the stable 50-instance public-suite rerun, parser rejection is rare for the full protocol: ARG reaches 92.0% schema well-formedness and ARG-Pro 98.0%. The dominant failure mode is missing

Table 4: Representative pre-ASP rejection cases.

Malformed trace	Parser failure	Decision
Missing verification body	NonEmpty fails because the required section is present only as a header.	Reject before grounding
Pattern appears after steps	Ord fails because section precedence is violated.	Reject before grounding
Verification cites "Step 6" but only Steps 1–4 exist	Ref fails because referential integrity is broken.	Reject before grounding

Table 5: Parser-facing interface statistics on public-benchmark runs. Strict-Parse measures correct section order with non-empty fields before enforcing verification references.

Method	N	Strict-Parse	WF	Missing/Empty	Ref-Fail
ARG-Pro	50	**98.0**	**98.0**	2.0	0.0
ARG	50	92.0	92.0	8.0	2.0

or empty required sections rather than order violations or broken references, which is consistent with treating the parser as an interface gate instead of a symbolic judge.

This separation matters because it prevents the verifier from treating malformed prose as if it were a valid symbolic object. Parser rejection therefore records an interface failure, not a solver-backed negative proof. Figure 7 makes the error shape explicit: the stable-suite parser burden is dominated by missing or empty sections, not by order corruption.

C Representative Alias and Marker Inventory

Grounding is deterministic but lexicon-dependent. Table 6 lists representative aliases and markers used in section-scoped extraction. The table is illustrative rather than exhaustive.

The grounding map is intentionally section-scoped: operation-style aliases are read from `Steps`, invariant-style markers from `Verification`, and family declarations from `Pattern`. This reduces, but does not eliminate, false positives from unconstrained keyword counting.

D Representative Contract Rules

The current verifier is implemented as ordinary ASP derivation rules rather than minimal unsatisfiable-core extraction. A representative family-local contract and verifier core is:

```
required_op(dynamic_programming, state_definition).
required_op(dynamic_programming, recurrence).
required_op(dynamic_programming, base_case).
required_inv(dynamic_programming, optimal_substructure).
required_inv(dynamic_programming, no_overlap).
missing_operation(T,Op) :- trace_declares(T,F),
required_op(F,Op),
not op_hit(T,Op).
missing_invariant(T,Inv) :- trace_declares(T,F),
required_inv(F,Inv),
not inv_hit(T,Inv).
verified(T) :- trace_declares(T,_), not
missing_operation(T,_),
not missing_invariant(T,_),
not bad_reference(T,_), not transition_violation(T,_).
```

This appendix fragment is representative rather than exhaustive, but it makes the rule object concrete: family-local facts specify obligations, and global verifier rules derive the diagnostic atoms used throughout the paper.

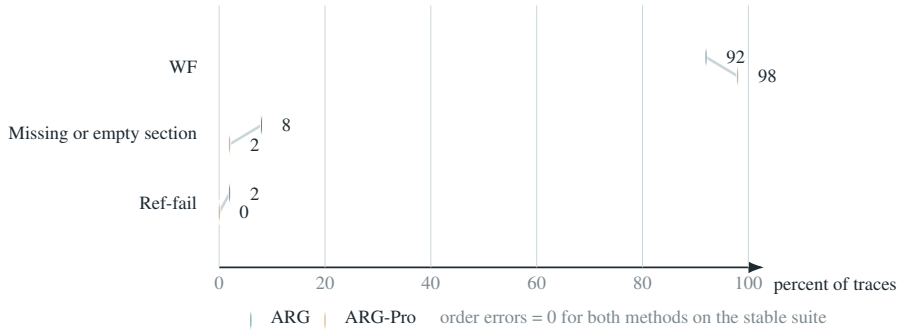


Figure 7: Parser-facing outcome profile on the stable public suite. The dominant rejection mode is missing or empty required sections, while order violations disappear entirely and reference failures remain rare.

Table 6: Representative section-scoped aliases and grounded atoms.

Family slice	**Representative surface form**	**Grounded atom kind**
Graph	"relax edge", "distance", "queue" in `Steps`	ops/inv: `edge`, `distance`, `reachability`
Dynamic programming	"base case", "recurrence", "dp[i]" in `Steps`	ops: `base_case`, `transition`, `state`
Data structures	"update", "query", "segment tree"	ops/inv: `update`, `query`, prefix consistency
String	"prefix function", "match", "failure link"	ops/inv: `pattern`, `prefix`, `failure_link`
Math/number theory	"gcd", "remainder", "congruence"	ops/inv: `gcd`, `equation`, `congruence`
Counting	"split into cases", "choose", "non-overlap"	ops/inv: `case_split`, `combination`, `disjoint_cases`
General search	"binary search", "window", "termination"	ops/inv: `binary_search`, `window`, `termination`

E Family Activation on the Stable Public Suite

Recomputing family declarations from stored ARG-family traces shows how concentrated contract coverage is on the stable 50-instance public suite. Most public arithmetic items never trigger a supported family at all, which is why applicability remains sparse even when the schema-level interface is clean.

The main implication is not that the contract library is ineffective, but that public benchmark questions seldom expose family evidence explicitly enough for the current grounding interface to bind. Figure 8 shows the same skew visually: most traces stay in the "no supported family" bucket, and the small activated remainder is concentrated in a narrow subset of the library rather than spread across all seven families.

F Grounding Robustness Under Paraphrase

To isolate grounding brittleness from generation quality, we apply structure-preserving paraphrases directly to stored ARG-family traces from the public external suite. The perturbation rewrites section

Table 7: Recomputed declared-family activation on the stable public-suite rerun (50 instances).

Family	ARG	ARG-Pro
none	42	42
dynamic_programming	4	3
counting	2	3
data_structure	1	1
string	1	0
general	0	1

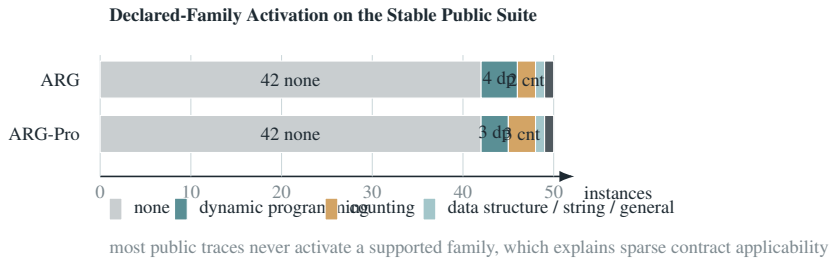


Figure 8: Stable-suite family activation for ARG-family traces. Sparse activation is dominated by the absence of any supported family declaration, with the remaining positive cases concentrated in dynamic-programming and counting-style contracts.

bodies in `Pattern`, `Steps`, and `Verification` while keeping headers, step indices, and final answers fixed, so any degradation reflects lexical grounding miss rather than parser failure or answer drift.

The result is asymmetric in exactly the way the paper claims: schema-level observability mostly survives, but family-grounded verification does not. On the stable public-suite rerun, WF preservation remains above 90% for both ARG and ARG-Pro, while the contract-level signal drops much faster once family names and operation markers are paraphrased (e.g., `BinarySearch` → `IntervalHalving`, `DynamicProgramming` → `MemoizedRecurrence`). Figure 9 makes that asymmetry visible: surface structure is relatively stable, but grounded contract evidence is lexicon-sensitive. This is evidence for a real robustness boundary rather than a speculative limitation.

G Paired Outcome Stability

Because the stable external-only suite contains only 50 instances, paired outcome counts should be read with small-sample caution. We therefore report both exact sign tests on discordant paired outcomes and a benchmark-balanced bootstrap over macro-accuracy differences. The resulting picture is supportive but intentionally conservative: ARG's paired edge over CoT is directionally consistent on both the full suite and the non-GSM8K slice, and the bootstrap intervals remain non-negative (strictly positive on the harder slice), but the exact two-sided sign test is still underpowered at this scale.

H Executable-Suite Family Breakdown

To check whether the executable-suite lift is carried by a single easy family, we report per-family answer accuracy across the six covered families. The result is mixed in the right way for a mechanistic claim: ARG-Pro does not exceed the best non-ARG baseline on already-saturated families, but it does recover meaningful margin on the harder combinatorial slices. CoinChange improves by +25.0 points over the best non-ARG baseline and Staircase by +5.0, yielding a +5.0 family-macro edge overall. This argues against a one-family artifact rather than for uniform dominance.

Table 8: Structure-preserving paraphrase stress test on stored ARG-family public-suite traces. The perturbation rewrites section bodies while keeping headers, step indices, and final answers fixed.

Method	N	WF Pres.	Ref Pres.	ASP App Before	ASP App After	Constraint Drop
ARG	50	92.0	98.0	8	6	75.0
ARG-Pro	50	98.0	100.0	8	6	100.0

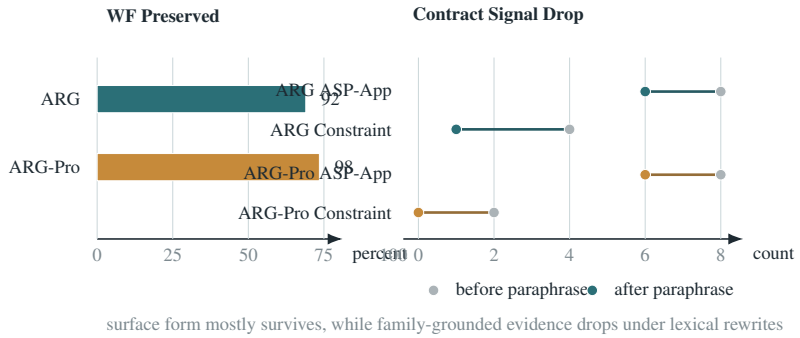


Figure 9: Paraphrase stress boundary on stored ARG-family traces. Schema-level well-formedness stays high, but contract-grounded applicability and constraint support fall once family names and operation markers are lexically rewritten.

I Selective Trust from Semantic Verification

The executable suite also lets us test whether a positive interface-side signal corresponds to a real trust region. Treating `semantic_verification_pass` as a selective gate yields near-ceiling conditional accuracy when it fires: 98.8% for ARG, 99.0% for ARG-Pro, and 98.9% for ARG-NoVerify. The residual non-pass bucket is dramatically harder, especially for ARG (17.5%) and ARG-NoVerify (24.0%). This is not yet a full reliability model, but it is strong evidence that the interface signal is decision-relevant rather than decorative.

J Authoring Surface and Incremental Family Extension

A recurring reviewer question is whether family coverage scales only by accumulating ad hoc global logic. In the current implementation it does not: family bundles are local additions over a fixed verifier core. Table 12 reports the present authoring surface, which ranges from 17 to 32 local assets per family with zero global verifier-rule edits. To make the marginal effect concrete, Table 13 replays a graph-family hold-out/add-back on stored GridSP traces. Removing the local bundle forces a 100.0% unknown rate by construction; restoring it recovers 100.0% declared-family support, 90.0% algorithm-constraint pass, and 100.0% ASP applicability on the same slice. Because this is a stored-trace replay, it isolates coverage gain rather than answer-generation gain.

K Directional Public-Slice Controls

To test whether answer accuracy and auditability can separate on public data, we also ran a small control rerun on the SVAMP slice from the larger external-only suite. We treat this slice as directional rather than headline evidence because it contains only 20 instances and several methods are at answer-level ceiling. The useful signal is orthogonal: removing verification or step indexing can preserve answer accuracy while collapsing the auditable surface.

This directional control is useful precisely because it is not an answer-level win story. On this public slice, the full protocol does not matter because it changes the answer set; it matters because it preserves an auditable trace when simpler variants do not. A complementary fixed-compute control on the same public slice compares a single-pass structured controller against selection-heavy baselines. ARG-Pro-NoExec matches the 100.0% answer ceiling with 858 average tokens and perfect structure/reference integrity, while JSON-CoT-BoN-Exec needs 1032.5 tokens at the same accuracy but

Table 9: Paired outcome and benchmark-balanced bootstrap summaries on the stable public-only suite. Macro Diff is candidate minus baseline in macro accuracy points; p-values use the exact two-sided sign test on discordant paired outcomes.

Candidate	Baseline / Slice	Wins	Losses	Macro Diff	Exact p
ARG	CoT (Full)	5	1	+8.02 [+0.00, +16.00]	0.2188
	CoT (Non-GSM8K)	5	0	+12.54 [+5.00, +22.50]	0.0625
ARG	JSON-CoT (Full)	3	2	+2.02 [-6.00, +10.00]	1.0000
	JSON-CoT (Non-GSM8K)	3	1	+4.98 [-5.00, +15.00]	0.6250
ARG	PlanSolve (Full)	3	1	+4.06 [-4.00, +12.00]	0.6250
	PlanSolve (Non-GSM8K)	2	0	+5.01 [+0.00, +12.50]	0.5000
ARG-Pro	ARG (Full)	1	2	-1.98 [-8.00, +4.00]	1.0000
	ARG (Non-GSM8K)	0	1	-2.50 [-7.50, +0.00]	1.0000

Table 10: Family-wise answer accuracy on the 120-instance executable suite. The last column compares ARG-Pro against the best non-ARG baseline on each family.

Family	N	ARG	ARG-Pro	Best baseline	Δ ARG-Pro
CoinChange	20	5.0	45.0	20.0	+25.0
GCD	20	100.0	100.0	100.0	+0.0
GridSP	20	100.0	100.0	100.0	+0.0
InsertIdx	20	65.0	100.0	100.0	+0.0
Staircase	20	65.0	100.0	95.0	+5.0
WindowSum	20	95.0	100.0	100.0	+0.0
Family-macro	–	71.7	90.8	85.8	+5.0

exposes no auditable surface, and PlanSolve-BoN-Exec falls to 85.0% accuracy despite three-node selection (Table 15). The point is again separation rather than universal dominance: extra selection compute does not by itself recover an auditable trace.

L Deterministic Interface Overhead

Stored-trace replay confirms that the deterministic checker stack is fast enough to be treated as a negligible wrapper around API generation rather than a second inference engine. The important qualifier is explicit: these timings exclude model API latency and therefore measure only parser validation, grounding, and ASP solving on already materialized traces.

M Submission-Facing Anonymous Artifact Surface

Supplementary material accompanying this submission includes an anonymous artifact skeleton that fixes the intended reproduction surface before any camera-ready packaging. The supplement is organized so that reviewers can reproduce both answer-level and process-level metrics without editing the codebase. It contains:

1. benchmark manifests, fixed seeds, and item metadata for the public external suite used in the main paper;
2. an asset inventory that records benchmark provenance, release status, and the checklist-facing license/terms summary state for each external asset;
3. prompt templates and method-specific strategy configurations for CoT, structured baselines, ARG, and ARG-Pro;
4. executable checkers, ASP contract files, and grounding utilities used to compute process-side metrics;
5. per-instance logs containing raw traces, parsed sections, normalized answers, grounded atoms, verifier outputs, and aggregate summaries; and

Table 11: Selective-trust behavior of the semantic verification signal on the 120-instance executable suite. A positive semantic-pass signal is highly precise for executable correctness, while the residual non-pass bucket remains much harder.

Method	Coverage	Sel-Acc	Non-pass Acc	Recall	Lift
ARG	66.7	98.8	17.5	91.9	+81.3
ARG-Pro	80.8	99.0	56.5	88.1	+42.5
ARG-NoVerify	79.2	98.9	24.0	94.0	+74.9

Table 12: Family-local authoring surface in the current contract library. Extending a family adds only local lexicon and contract assets; global verifier rules stay unchanged.

Family	Pattern	Step	Verif.	ReqOps	ReqInvs	Alias	Total
Graph	14	8	5	3	2	0	32
Counting	6	5	4	3	2	4	24
DataStruct	8	7	4	2	2	1	24
DynamicProg	5	6	4	3	2	2	22
String	6	6	3	2	2	3	22
General	5	5	4	2	2	2	20
Math	4	5	4	2	2	0	17

6. a step-by-step reproduction guide plus figure/table generation scripts that rebuild the reported artifacts from stored outputs, including parser-rejection, interface-overhead, paired-significance, and failure-bucket summaries.

The current supplement is intentionally narrower than a final bundled artifact release: model credentials are excluded, and a final anonymous bundle should still carry verified third-party license/terms text for each released external asset. Reproduction instructions nevertheless target the same command-line entry points used to generate the stored run summaries.

Table 13: Stored-trace hold-out/add-back replay for the graph family bundle on the 20-instance GridSP slice under ARG. The add-back restores audit coverage without any global verifier-rule changes.

State	Declared	Alg-pass	ASP-app	Unknown
Hold-out	0.0	0.0	0.0	100.0
Add-back	100.0	90.0	100.0	0.0

Table 14: Directional control evidence on the public SVAMP slice (20 fixed instances). All three variants solve the slice at ceiling answer accuracy, but auditability metrics collapse when verification or explicit steps are removed.

Method	Acc	Ver-Cov	WF	Ref
ARG	100.0	100.0	100.0	100.0
ARG-NoVerify	100.0	0.0	0.0	0.0
ARG-NoSteps	100.0	100.0	0.0	0.0

Table 15: Directional fixed-compute public controls on the 20-instance SVAMP slice. This slice is not a headline answer-level benchmark; it isolates whether selection-heavy baselines preserve auditability at similar or higher token budgets.

Method	Acc	AvgTok	Nodes	Struct	WF	Ref
ARG-Pro-NoExec	100.0	858.0	1.0	1.00	100.0	100.0
JSON-CoT-BoN-Exec	100.0	1032.5	3.0	0.60	0.0	0.0
PlanSolve-BoN-Exec	85.0	1026.0	3.0	0.19	0.0	0.0

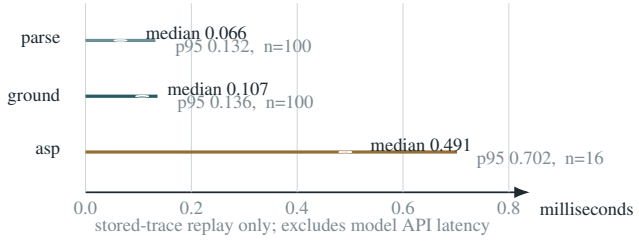


Figure 10: Deterministic checker overhead on stored public-suite traces. All three stages remain sub-millisecond at median latency, and even the ASP stage stays comfortably below one millisecond at p95 on applicable items.

References

Gerhard Brewka, Thomas Eiter, and Mirosław Truszczyński. Answer set programming at a glance. *Communications of the ACM*, 54(12):92–103, 2011.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

Ning Dai, Zheng Wu, Renjie Zheng, Ziyun Wei, Wenlei Shi, Xing Jin, Guanlin Liu, Chen Dun, Liang Huang, and Lin Yan. Process supervision-guided policy optimization for code generation. *arXiv preprint arXiv:2410.17621*, 2024.

Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

Gabriel Freedman, Adam Dejl, Deniz Gorur, Xiang Yin, Antonio Rago, and Francesca Toni. Argumentative large language models for explainable and contestable claim verification. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(14):14930–14939, 2025.

Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Multi-shot asp solving with clingo. *Theory and Practice of Logic Programming*, 19(1):27–82, 2019.

Michael Gelfond and Vladimir Lifschitz. The stable model semantics for logic programming. In *Logic Programming, Proceedings of the Fifth International Conference and Symposium*, volume 2, pages 1070–1080. MIT Press, 1988.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured nlp tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, 2023.

Adam Ishay, Zhun Yang, and Joohyung Lee. Leveraging large language models to generate answer set programs. In *Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning*, pages 374–383, 2023.

Alon Jacovi and Yoav Goldberg. Towards faithfully interpretable nlp systems: How should we define and evaluate faithfulness? In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 4198–4205, 2020.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The twelfth international conference on learning representations*, 2023.

Zachary C Lipton. The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery. *Queue*, 16(3):31–57, 2018.

Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Meiqi Guo, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, Lei Meng, et al. Improve mathematical reasoning in language models by automated process supervision. *arXiv preprint arXiv:2406.06592*, 2024.

Qing Lyu, Marianna Apidianaki, and Chris Callison-Burch. Towards faithful model explanation in nlp: A survey. *Computational Linguistics*, 50(2):657–723, 2024.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.

Katie Matton, Robert Osazuwa Ness, John Guttag, and Emre Kıcıman. Walk the talk? measuring the faithfulness of large language model explanations. *arXiv preprint arXiv:2504.14150*, 2025.

Theo Olausson, Alex Gu, Ben Lipkin, Cedegao Zhang, Armando Solar-Lezama, Joshua Tenenbaum, and Roger Levy. Linc: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5153–5176, 2023.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3806–3824, 2023.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. " why should i trust you?" explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 1135–1144, 2016.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 2609–2634, 2023.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. Satlm: Satisfiability-aided language models using declarative prompting. *Advances in Neural Information Processing Systems*, 36:45548–45580, 2023.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.

Yuqicheng Zhu, Nico Potyka, Daniel Hernández, Yuan He, Zifeng Ding, Bo Xiong, Dongzhuoran Zhou, Evgeny Kharlamov, and Steffen Staab. Argrag: Explainable retrieval augmented generation using quantitative bipolar argumentation. In *Proceedings of The 19th International Conference on Neurosymbolic Learning and Reasoning*, volume 284 of *Proceedings of Machine Learning Research*, pages 697–718, 2025.